

Trabalho 3º Bimestre - Desenvolvimento de uma Aplicação Full Stack

1. Objetivo

Agilizar e ampliar o trabalho em equipe, o versionamento de código e desenvolvimento em paralelo usando **github é obrigatório**. Sua equipe de desenvolvimento deve usar metodologias ágeis e criar uma *sprint* (30 dias, ou duas de 15 dias) de trabalho. Apresentar no relatório final com as tarefas delegadas a cada integrante da equipe, e o histórico diário de *commits* de cada usuário no **github**.

Observações mandatárias:

1. Não serão permitidos trabalhos individuais ou em duplas;
2. Não serão permitidos trabalhos que não usem o github como ferramenta de versionamento;
3. Não serão permitidos trabalhos que não usem o docker para modularização (back-end, front-end e BD(PostgreSQL));
4. Não serão permitidos trabalhos que não usem o python + flask + WSGI Unicorn no back-end;
5. As equipes devem ter no mínimo 3 integrantes e no máximo 4 (destes, haverá um líder técnico responsável pela divisão das tarefas diárias e liderança técnica de engenharia de software do projeto).

Desenvolver uma aplicação **full stack** capaz de consultar uma base de dados contendo informações de **servidores ativos e aposentados do Poder Executivo Federal**, apresentando os resultados de forma organizada ao usuário. A aplicação deverá ser modularizada utilizando **Docker**, sendo obrigatória a utilização de **Dockerfile** (para configurar o necessário de cada contêiner) ou **Docker Compose** (para orquestrar a comunicação dos contêineres). Construa a Dockerfile de cada máquina e/ou o Docker-compose.yaml já com os 3 containers (**DB-server**, **Back-end** e **Front-end**) necessários para o desenvolvimento da aplicação.

2. O contêiner db-server e as bases de dados de servidores do Executivo Federal

As bases de dados utilizadas pela aplicação serão provenientes do **Portal Brasileiro de Dados Abertos (dados.gov.br)**, contemplando os conjuntos de dados:

- **Gestão de Pessoas do Executivo Federal — Aposentados;**

<https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal--aposentados>

- **Gestão de Pessoas do Executivo Federal — Carreiras/Cargos.**

<https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal---carreiras--cargos>

Acesse o Dicionário de dados disponível no site do dados.gov.br, **organize e sintetize em um Sistema Gerenciador de Banco de Dados (SGBD - PostgreSQL)** para utilização pela aplicação conforme ensinado em sala de aula. Otimize o banco de dados para realização de buscas indexadas e por similaridade. A equipe deverá realizar o tratamento e a preparação dos dados necessários para sua importação e consulta no banco de dados.

O contêiner **db-server** deverá disponibilizar o SGBD utilizado pela aplicação, sendo obrigatória a utilização do **PostgreSQL**.

As bases de dados deverão permitir consultas relacionadas aos registros de servidores, incluindo informações como **nome, cargo/profissão, estado e instituição/órgão**, conforme os campos disponíveis nos conjuntos de dados utilizados.

3. O contêiner do Back-end / API

A aplicação deverá disponibilizar **endpoints** para realizar as consultas na base de dados. Exemplo:

GET /api/servidores?cargo=<cargo>

GET /api/servidores?uf=<uf>

GET /api/servidores?orgao=<orgao>

GET /api/servidores?nome=<nome>&cargo=<cargo>&uf=<uf>

GET /api/servidores?similar="<%no%me%"&limit=100

(sendo busca por similaridade e **100** o número máximo de pessoas com nome similar)

Buscas possíveis:

- Busca por **nome**;
- Busca por **cargo/profissão**;
- Busca por **estado (UF)**;
- Busca por **instituição/órgão**;
- Combinação de diferentes filtros de busca;
- Envio dos dados dos registros encontrados no formato **JSON**.

O back-end deverá ser desenvolvido **obrigatoriamente** utilizando:

- **Python**
- **Flask + WSGI Unicorn** (multi-threading atendendo mais requisições dos usuários)

Os endpoints deverão retornar os dados em formato **JSON**.

A API do backend deverá tratar situações como:

- Strings inválidas;
- Tentativas de comandos sql (*sql injection*);
- Nome não encontrado;
- Nome retornando múltiplos registros;
- Cargos inválidos;
- UF inválidas;
- Instituição/Órgão inválidos;
- Página inexistente;
- Tempo de busca excedendo o timeout;
- Excesso de buscas ao mesmo tempo (limitar de acordo com a plataforma de execução);
- Parâmetros de busca inválidos.

4. Front-end

1. Aplicação Web

Desenvolvimento de uma interface web utilizando:

- HTML/CSS/JavaScript; (**faça um design estiloso**)
- Escolha do *framework* de desenvolvimento *web* liberado para a equipe de desenvolvimento: exemplos: *Flask, Django, Node.js, React, Vue, Angular, Svelte*, entre outros.

Adicional: Aplicação Desktop sem docker

Desenvolvimento de uma aplicação desktop utilizando:

- Python
- PyQt/PySide, tkinter ou toga BeeWare (multi-plataforma)
- Compile sua aplicação final

Em ambos os casos, os apps deverão consumir a API Flask do *backend* via requisições HTTP/HTTPS.

5. Funcionalidades obrigatórias (requisitos funcionais)

A aplicação deverá possuir, no mínimo:

- **Busca por nome (exato):** O usuário informa um nome e o sistema deve consultar a API e apresentar os registros encontrados.
- **Busca por similaridade de nome:** O usuário informa um nome “%NOME%SOBRENOME%”, “NOME SOBRENOME%”, “NOME%SOBRENOME”, e um limite máximo de nomes resultantes.
- **Busca por cargo/profissão:** O usuário informa um cargo ou profissão e o sistema deve consultar a API.
- **Busca por estado:** O usuário informa uma unidade federativa (UF) e o sistema deve consultar a API.
- **Busca por instituição/órgão:** O usuário informa uma instituição ou órgão e o sistema deve consultar a API.
- **Busca combinada avançada:** O usuário poderá utilizar mais de um critério simultaneamente, como cargo e estado ou instituição e estado.

Caso exista apenas um resultado, seus dados deverão ser apresentados. Caso existam vários resultados, eles deverão ser apresentados de maneira paginada, permitindo ao usuário navegar pelos resultados.

6. Entrega Limite(13/10/2026): Não serão aceitos trabalhos entregues após a data limite

Toda a aplicação deve estar no **github** e seu histórico deve estar documentado. Quais são as tarefas diárias de cada desenvolvedor da equipe e quais foram os **commits** de cada desenvolvedor da equipe. O líder técnico deve fornecer um gráfico de **burndown** das tarefas desenvolvidas pela equipe, bem como os pontos fortes da equipe e os pontos fracos, principais dificuldades e impedimentos encontrados ao longo do desenvolvimento do projeto.

Os alunos deverão entregar um arquivo **.zip** contendo a estrutura de pastas e arquivos apresentada abaixo, além de quaisquer outros arquivos necessários para a execução do projeto (exceto o BD disponibilizado). O projeto deverá possuir uma pasta principal denominada projeto. Dentro dela, deverão existir três subpastas: **backend**, **frontend** e **db**, além do arquivo **docker-compose.yml**.

Os contêineres do **backend** e do **frontend** deverão ser configurados por meio de seus respectivos **Dockerfiles**. A aplicação deverá ser configurada de forma que possa ser executada a partir da raiz do projeto, utilizando o comando: **docker compose up –build**. O objetivo é que, após a execução do comando, os serviços necessários para o funcionamento da aplicação sejam inicializados automaticamente e a aplicação fique pronta para uso.

7. Avaliação

Durante a avaliação serão feitos testes exaustivos de busca na API do **backend**. Realizem testes de sobrecarga paralela para garantir que o sistema não entre em colapso e trave. Crie um mecanismo de segurança para garantir e evitar que isso não ocorra. Descreva as limitações encontradas no relatório final.

1. Relatório de divisão das tarefas da equipe e histórico do uso diário do **github** (2,0 pontos);
2. Funcionamento da inicialização dos contêineres do **docker** via comando solicitado (1,0 pontos);
3. Funcionamento do **backend** com **python+flask+WSGI Unicorn** (4,0 pontos);
4. Funcionamento do frontend (aplicação web apresentando os dados adequadamente) (3,0 pontos)
5. Adicional (1,0 pontos)
 - a. Desenvolvimento de aplicativo **Desktop** com interface gráfica (1,0 pontos)